

JavaScript Fundamentals — Study Notes

1. Variables (let/const), Data Types, Operators, Type Coercion

Variables

- `let` — value change ho sakti hai (reassignable), block-scoped.
- `const` — reassign nahi ho sakta, block-scoped. Object/array ke andar ki properties change ho sakti hain (reference fixed hai, content nahi).
- `var` — purana way, function-scoped, hoisting ke masle deta hai. Avoid karo modern code mein.

```
js
let age = 22;
age = 23; // ok

const name = "Momin";
// name = "Ali"; // Error: Assignment to constant variable
```

Data Types

Primitive types:

- `string` — text: `"hello"`
- `number` — integers aur decimals dono: `42`, `3.14`
- `boolean` — `true` / `false`
- `undefined` — variable declare hua but value assign nahi hui
- `null` — intentional "no value"
- `symbol` — unique identifiers ke liye
- `bigint` — bohot bari integers ke liye

Non-primitive:

- `object` — key-value pairs
- `array` — object hi hai, but ordered list
- `function` — bhi ek object type hai

js

```
typeof "hello"; // "string"
typeof 42;       // "number"
typeof true;     // "boolean"
typeof undefined; // "undefined"
typeof null;     // "object" <- famous JS quirk/bug
typeof {};      // "object"
typeof [];      // "object"
```

Operators

- Arithmetic: `+ - * / % **`
- Assignment: `= += -= *= /=`
- Comparison: `== != === !== > < >= <=`
- Logical: `&& || !`
- Ternary: `condition ? valueIfTrue : valueIfFalse`

`==` vs `===` (Type Coercion)

- `==` (loose equality) values ko compare karne se pehle type convert (coerce) kar deta hai.
- `===` (strict equality) type aur value dono check karta hai — koi coercion nahi.

js

```
"5" == 5;    // true  (string "5" number 5 mein convert hoti hai)
"5" === 5;   // false (types different hain)

0 == false;  // true
0 === false; // false

null == undefined; // true
null === undefined; // false
```

Rule of thumb: Hamesha `===` use karo, `==` avoid karo — predictable behavior milta hai.

Type Coercion examples (common gotchas)

```
js

"5" + 3;    // "53"  (number string mein convert hota hai, concatenation)
"5" - 3;    // 2     (string number mein convert hoti hai, subtraction)
"5" * "2";  // 10
true + 1;   // 2     (true -> 1)
false + 1;  // 1
```

Active Recall — Section 1

1. `let` aur `const` mein farak kya hai?
 2. `typeof null` kya return karta hai, aur kyun ye JS ka bug consider hota hai?
 3. `==` aur `===` mein kya difference hai? Konsa use karna chahiye aur kyun?
 4. `"5" + 3` ka result kya hoga aur kyun?
 5. `var` ko modern JS mein avoid kyun kiya jata hai?
-

2. Conditionals, Loops, and Functions

Conditionals

```
js

if (age >= 18) {
  console.log("Adult");
} else if (age >= 13) {
  console.log("Teenager");
} else {
  console.log("Child");
}

// switch statement
switch (day) {
  case "Mon":
    console.log("Start of week");
    break;
  default:
    console.log("Other day");
}
```

Loops

```
js
```

```
// for loop
for (let i = 0; i < 5; i++) {
  console.log(i);
}

// while loop
let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}

// do-while (kam se kam ek baar chalta hai)
let j = 0;
do {
  console.log(j);
  j++;
} while (j < 5);

// for...of – iterable values (array/string) ke liye
for (const val of [10, 20, 30]) {
  console.log(val);
}

// for...in – object keys ke liye
for (const key in { a: 1, b: 2 }) {
  console.log(key);
}
```

Functions — 3 tareeke

1. Function Declaration — hoisted hoti hai (upar call kar sakte ho, define baad mein).

```
js

function greet(name) {
  return `Hello, ${name}`;
}
```

2. Function Expression — variable mein assign hoti hai, hoisted nahi hoti.

```
js

const greet = function (name) {
  return `Hello, ${name}`;
};
```

3. Arrow Function — short syntax, `this` ko apne surrounding scope se leti hai (lexical `this`), khud ka `this` nahi banati.

```
js

const greet = (name) => {
  return `Hello, ${name}`;
};

// single expression ho to implicit return
const greet2 = (name) => `Hello, ${name}`;

// single parameter ho to parentheses optional
const square = x => x * x;
```

Important difference: Regular function apna `this` define karti hai (kaise call hui uspe depend karta hai), arrow function `this` ko outer/lexical scope se inherit karti hai — isliye arrow functions object methods ya class methods mein `this` binding issues avoid karne ke liye kaam aati hain.

Active Recall — Section 2

1. `for...of` aur `for...in` mein kya farak hai?
 2. Function declaration aur function expression mein hoisting ka farak kya hai?
 3. Arrow function normal function se `this` ke mamle mein kaise different hai?
 4. `do-while` loop normal `while` se kaise different hai?
 5. Implicit return kya hota hai arrow functions mein?
-

3. Arrays and Objects; Key Array Methods

Arrays

```
js
const fruits = ["apple", "banana", "mango"];
fruits[0];           // "apple"
fruits.length;       // 3
fruits.push("kiwi"); // end mein add
fruits.pop();         // end se remove
fruits.shift();       // start se remove
fruits.unshift("x");  // start mein add
```

Objects

```
js
```

```
const person = {
  name: "Momin",
  age: 22,
  greet() {
    return `Hi, I'm ${this.name}`;
  }
};

person.name;      // dot notation
person["name"];   // bracket notation (dynamic keys ke liye zaroori)

// spread operator – copy/merge
const updated = { ...person, age: 23 };
```

Key Array Methods (bohot important — interview aur daily use dono)

forEach — har element pe kuch karo, kuch return nahi hota (undefined).

```
js

[1, 2, 3].forEach(num => console.log(num * 2));
```

map — har element ko transform karke NAYI array return karta hai.

```
js

const doubled = [1, 2, 3].map(num => num * 2);
// [2, 4, 6]
```

filter — condition true karne wale elements ki NAYI array return karta hai.

```
js
```



```
const evens = [1, 2, 3, 4].filter(num => num % 2 === 0);  
// [2, 4]
```

find — pehla element jo condition match kare, return karta hai (single value, array nahi).

```
js  
  
const found = [1, 2, 3, 4].find(num => num > 2);  
// 3
```

reduce — pori array ko ek single value mein "reduce" karta hai (sum, object, etc).

```
js  
  
const sum = [1, 2, 3, 4].reduce((acc, curr) => acc + curr, 0);  
// 10 (acc = accumulator, 0 = initial value)  
  
// object banane ke liye bhi use hota hai  
const counts = ["a", "b", "a"].reduce((acc, item) => {  
  acc[item] = (acc[item] || 0) + 1;  
  return acc;  
}, {});  
// { a: 2, b: 1 }
```

Quick comparison:

Method	Returns	Use case
forEach	undefined	side effects (logging, DOM update)
map	new array (same length)	transform each element
filter	new array (subset)	select matching elements
find	single element / undefined	first match dhoondna
reduce	any single value	sum, group, flatten, custom logic

Active Recall — Section 3

1. `map` aur `forEach` mein farak kya hai?
2. `filter` aur `find` mein farak kya hai?
3. `reduce` ka accumulator aur initial value kya role play karte hain?
4. Object ki property access karne ke do tareeke kya hain, aur bracket notation kab zaroori hoti hai?
5. Spread operator (`...`) object copy karne mein kaise madad karta hai?

4. The DOM — Selecting, Creating, Updating, Removing Elements

Selecting Elements

```
js
document.getElementById("myId");
document.querySelector(".myClass"); // pehla match
document.querySelectorAll(".myClass"); // saare matches (NodeList)
document.getElementsByTagName("div");
document.getElementsByClassName("box"); // live HTMLCollection
```

`querySelector` / `querySelectorAll` most flexible hain — CSS selectors accept karte hain.

Creating Elements

```
js

const div = document.createElement("div");
div.textContent = "Hello";
div.classList.add("box");
document.body.appendChild(div);
```

Updating Elements

```
js

element.textContent = "New text";           // plain text
element.innerHTML = "<b>Bold</b>";           // HTML parsehota hai (XSS risk agar
element.setAttribute("src", "img.png");
element.classList.add("active");
element.classList.remove("hidden");
element.classList.toggle("open");
element.style.color = "red";
```

Removing Elements

```
js

element.remove();                           // modern, direct
parent.removeChild(element);                 // older way
```

Active Recall — Section 4

1. `querySelector` aur `getElementById` mein kya farak hai?
2. `textContent` aur `innerHTML` mein kya difference hai, aur `innerHTML` security ke lihaz se kyun risky hai?
3. Element create karke page mein add karne ke steps kya hain?

4. `classList.toggle` kya karta hai?
 5. Element remove karne ke do tareeke kya hain?
-

5. Events — Listeners, Event Object, Bubbling, Delegation

Event Listeners

```
js

button.addEventListener("click", function (event) {
  console.log("Clicked!");
});

// arrow function version
button.addEventListener("click", (e) => {
  console.log(e.target); // jis element pe click hua
});

// remove karna ho to named function chahiye
function handleClick(e) { console.log("clicked"); }
button.addEventListener("click", handleClick);
button.removeEventListener("click", handleClick);
```

The Event Object

Har event listener ko ek `event` object milta hai jo useful info deta hai:

```
js
```

```
button.addEventListener("click", (e) => {
  e.target;           // jis element pe actually click hua
  e.currentTarget;    // jis element pe listener attach hai
  e.preventDefault(); // default behavior rokna (e.g. form submit, link navigation)
  e.stopPropagation(); // bubbling rokna
  e.type;              // "click"
});
```

Event Bubbling

- Jab kisi child element pe event fire hota hai, wo upar apne parents tak "bubble" karta hai.
- Matlab agar `div > button` hai aur dono pe click listener laga ho, to button click karne se dono listeners challenge (pehle button ka, phir div ka).

html

```
<div id="parent">
  <button id="child">Click</button>
</div>
```

js

```
parent.addEventListener("click", () => console.log("Parent clicked"));
child.addEventListener("click", () => console.log("Child clicked"));

// button click karne pe output:
// "Child clicked"
// "Parent clicked"
```

Event Delegation

- Bubbling ka fayda uthate hue, ek hi listener parent pe laga do, aur `e.target` check karke pata lagao konsa child clicked hua.
- Faida: dynamically add hone wale elements pe bhi kaam karta hai, aur performance behtar hoti hai (bohot saare listeners lagane ki bajaye ek hi listener).

```
js
document.getElementById("list").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") {
    console.log("Clicked item:", e.target.textContent);
  }
});
```

Active Recall — Section 5

1. Event bubbling kya hota hai, example do.
2. `e.target` aur `e.currentTarget` mein farak kya hai?
3. `e.preventDefault()` kab use karte hain?
4. Event delegation kya hai aur ye faidemand kyun hai?
5. Event listener remove karne ke liye named function kyun zaroori hoti hai (anonymous function se kyun nahi hota)?

6. Debugging with Console and Browser DevTools

Console Methods

```
js
```

```
console.log("normal output");
console.warn("warning message");
console.error("error message");
console.table([{a:1,b:2},{a:3,b:4}]); // tabular data
console.group("Group name");
console.log("nested log");
console.groupEnd();
```

Browser DevTools (Chrome/Firefox — F12 ya Right Click > Inspect)

- **Elements tab** — live DOM/HTML aur CSS inspect/edit karna.
- **Console tab** — logs dekhna aur JS commands run karna.
- **Sources tab** — breakpoints laga kar step-by-step code execution dekhna (debugger).
- **Network tab** — API calls, requests/responses, loading time dekhna.
- **Application tab** — localStorage, cookies, sessionStorage inspect karna.

Debugger Statement & Breakpoints

```
js

function calculate(a, b) {
  debugger; // execution yahan pause ho jayega jab devtools open ho
  return a + b;
}
```

- Sources tab mein line number pe click karke bhi breakpoint laga sakte ho, bina code mein `debugger` likhe.
- Breakpoint pe pahunch kar variables ki current values inspect kar sakte ho, aur step-over/step-into se line-by-line execution dekh sakte ho.

Common Debugging Tips

- `console.log` variables ke pass rakho taake pata chale execution kahan tak pohuncha aur values kya thi.
- Error messages ko pura parho — line number aur file batata hai kahan issue hai.
- `typeof` use karke variable ka actual type check karo agar unexpected behavior ho raha ho.

Active Recall — Section 6

1. `console.table` kis situation mein useful hota hai?
2. `debugger` statement kya karta hai?
3. Network tab kis cheez ke liye use hota hai?
4. Breakpoint code mein likhe bagair kaise laga sakte hain?
5. Application tab mein kya inspect kar sakte hain?